Lab8 LCD Software Plan

Tuesday, March 24, 2026 9:30 AM

Pulse enable → Clock in bits

↓

Write Command → Set DB Lines, then Clock

↓

Init LCD - Send Command Sequence

Write Char → Commands w/ Values

↓

Print String → Chars

ECEG 247: Lab #8 Interfacing the 1602 LCD

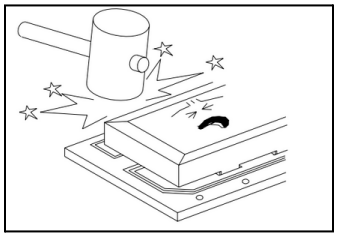
It is time to start building up our own C libraries that make the MSP430 work for us. In this lab, we are moving beyond using pre-written code and learning to interface directly with hardware. This is really where the hardware and software intersect and have to be co-designed and considered together. You will need to read and make sense of a manufacturer's datasheet, interface a 16x2 Liquid Crystal Display (LCD) to your microcontroller (using wires and a breadboard), and write your own software library to display text. In other words: We finally get to do something that does not involve blinking an LED! Yay!

If you already know exactly how to write an LCD driver from scratch, well grab the datasheet and get to work. If not, this set of notes will guide you through it. By the end of this lab and the next few lectures, you should understand:

1. How to read electrical and timing info from a standard (and sometimes imperfect) datasheet.
2. How to interface physical hardware with microcontroller software.
3. The concept of "clocking" parallel data to another device.
4. How to build a reusable, modular C library.

Part 1: Datasheet Scavenger Hunt

Before plugging in any wires, we need to understand the hardware. We are using the 16-character, 2-line LCD module. These are commonly referred to as a 1602 LCD and are almost all based on a Hitachi HD44780 chip. The overall LCD modules provide a simpler interface to the underlying Hitachi chip. If you search for “1602 LCD datasheet filetype:pdf” you will find many. Typically they all describe the underlying functionality. The datasheet we will be using is available on the Moodle folder. The file “datasheet_TC1602A-01T.pdf” is a fairly decent datasheet (I’ve seen much worse) and would be the one I would use. The other one “LCD1602.pdf” is a bit harder to read and mostly included because the cartoon warnings about treating your LCD well made are worth opening it just to look at! So, open your datasheet of choice and answer the following questions.



A quick warning: Engineering datasheets can sometimes have contradictions or require you to read the fine print!

1. Powering the Logic

- What does the datasheet say is the required operating voltage for the logic of this LCD? Which pins are used for power and ground? Answer this, note the section, and also include a small screenshot of this.
- 5v
- **DC ELECTRICAL CHARACTERISTICS**

Parameter	Symbol	Condition	Min	T_{YP}	Max	Unit
Supply voltage for logic	VDD	--	4.8	5.0	5.2	V
Supply current for logic	IDD	--	--	120	150	mA
Operating voltage for LCD	VLCD	-10℃				
		25℃	4.8	5.0	5.2	V
		+60℃				
Input voltage "H" level	VIH	--	0.7 VDD	--	VDD+0.3	V
Input voltage "L" level	VIL	--	0	--	0.2VDD	V

> ***A note:*** *While the datasheet might say 5 V, our specific boards actually work fine with 3.3 V logic levels. This is great news since that is what our MSP430 uses! However, the backlight and the contrast are very difficult to see without 5 V. The LCD will be a bit dim and difficult to see when powered only from 3.3 V, but it is possible that you can use a 5 V supply from your Analog Discovery to power the LCD module. Keep this in mind when planning your wiring.*

2. Contrast Control

- Look at the "Interface Pin Functions" and "Connection with MCU" sections. Pin 3 is labeled "V0". What is this pin used for? What specific hardware component does the datasheet recommend you connect to it to adjust this value?

"Adjusting supply voltage for LCD driving." (contrast) - Potentiometer

3. The Backlight Trap (Read Carefully!)

- Look at the table for "Interface Pin Functions". What voltage does it say to apply to Pin 15 (LED+)?

5v through resistor to limit current

Pin No.	Symbol	Level	Description
1	VSS	0V	Ground.
2	VDD	+5.0V	Power supply for logic operating.
3	V0	--	Adjusting supply voltage for LCD driving.
4	RS	H/L	A signal for selecting registers: 1: Data Register (for read and write) 0: Instruction Register (for write), Busy flag-Address Counter (for read).
5	R/W	H/L	R/W = "H": Read mode. R/W = "L": Write mode.
6	E	H/L	An enable signal for writing or reading data.
7	DB0	H/L	This is an 8-bit bi-directional data bus.
8	DB1	H/L	
9	DB2	H/L	
10	DB3	H/L	
11	DB4	H/L	
12	DB5	H/L	
13	DB6	H/L	
14	DB7	H/L	
15	LED+	+5.0V	Power supply for backlight.
16	LED-	0V	The backlight ground.

- Now, look at the bottom of the "LED Backlight Characteristics" table. What does the warning **NOTE** explicitly tell you *not* to do?

"NOTE: Do not connect +5V directly to the backlight terminals. This will ruin the backlight."

- *Your task:* You need to determine how to safely wire this when using a 5 V supply! What is your solution?

4. Communication Pins

- **RS (Register Select):** What happens if you send a `0` (Low) to this pin? What happens if you send a `1` (High)?

0 = Instruction Register

1 = Data Register

- **R/W (Read/Write):** For this entire lab, we only want to *write* data to the LCD. How should this pin be wired permanently to save us a GPIO pin on the MSP430?

Pin 5 LOW

Part 2: Hardware Wiring

You have the choice of wiring this LCD in **8-bit mode** (using all data pins DB0-DB7) or **4-bit mode** (using only DB4-DB7 to save pins on your microcontroller). 4-bit mode takes less wiring but slightly more complex code. The choice is yours. My warning/recommendation would be that 8-bit mode uses more wiring, but tends to be slightly easier to debug. On the other hand, 4-bit mode feels a bit more fun, but tends to either work fine, or not work at all. I started with 8-bit mode.

Wiring Checklist:

- [X] **Make a wiring plan**: Given the amount of wiring you are going to need to do, take a moment to plan this out. Where are you going to put this on your breadboard? What pins are you going to need to use on the MSP430? How many wires will you need to grab? Draw a rough sketch and note down the pins you plan to use.
- [X] **Power & Ground:** Wire up VSS and VDD. Remember our discussion about 3.3 V vs 5 V! It is possible to run this whole module from 3.3 V supply of the Launchpad, but will be a bit difficult to see.
- [X] **Contrast:** Connect your potentiometer circuit to VDD, Ground, and the wiper to V0 on the LCD.
- [X] **Read/Write:** Wire the R/W pin to the location you planned.
- [X] **Backlight:** Connect LED- to Ground, and LED+ to the appropriate power pin or circuit you designed.
- [X] **Data & Control:** Connect RS, E, and your chosen Data pins to your MSP430 GPIO pins. Make sure your physical wiring and your wiring plan notes are all in sync since you'll need this when we code things.

VCC -> + Rail (Female->Male)

GND -> - Rail (Male->Male)

+ Rail -> VDD, LED+ (BLA) (THRU 10k RESISTOR) (Male->Male)

- Rail -> VSS, LED- (BLK) (Male->Male)

Potentiometer (25k) -> VDD (Male->Male)

|

-> V0 (Male->Male)

|

-> - Rail (Male->Male)

RW -> GND (Write mode) (Male->Male)

P1.0 -> D0 (Male->Male)

P1.1 -> D1 (Female->Male)

P1.2 -> D2

P1.3 -> D3

P1.4 -> D4

P1.5 -> D5

P1.6 -> D6

P1.7 -> D7

P2.0 -> RS

P2.1 -> E

Part 3: How to talk to things: "Clocking" in Data

Look at the **"Write Mode Timing Diagram"** in the AC Characteristics section of the datasheet. This is the most critical concept in this lab to getting your LCD to operate properly.

The LCD has parallel data lines (`DB0` – `DB7`). You can place the correct `1`s and `0`s on these lines all you want, but the LCD will completely ignore them until you explicitly tell it to "read" these lines. This is done using the **E (Enable)** pin to “clock” in and read the data.

The Clocking Sequence
This is shown in the figure on the AC Characteristics page, but may not be obvious. There are a lot of timing values specified in nanoseconds related to digital state transitions. The steps to understand are as follows—

1. **Set the Mode:** Set the RS pin High (for text) or Low (for a command).
2. **Present the Data:** Set your DB pins High or Low to represent the binary value of the character or command you want to send.
3. **Prime the Clock:** Set the E pin HIGH.
4. **Wait:** You must leave the E pin HIGH for a short duration. (Look up the minimum "E Pulse Width" in the datasheet). If your MSP430 is using a 1 MHz clock, how many instructions (or cycles) does E need to remain high?
5. **Clock it in:** Pull the E pin LOW. *The exact moment the E pin transitions from High to Low (the falling edge) is when the LCD "clocks in" and saves the parallel bits.*

Checkpoint: Before starting to write your main code, sketch out a flowchart or write a pseudo-code function called `pulseEnable()` that performs steps 3, 4, and 5. Show this and your wiring to the instructor.

Part 4: Building the Library

Over the rest of this lab and the upcoming lecture periods, you will construct a modular C library. Because we want this to be good software engineering, don't just dump everything in `main()`.

Your goal is to write the following functions:

1. **`initLCD(void)`:**
    - Sets the appropriate MSP430 pins you are using to be outputs.
    - Sends the required initialization sequence (found in the datasheet) to wake up the LCD.
    - The specific initial procedure is different depending on whether you are using 4-bit mode or 8-bit mode.
2. **`writeCommand(unsigned char command)`:**
    - Clears the RS pin to LOW.
    - Places the bits of the `command` onto the DB pins.
    - Calls your `pulseEnable()` function to clock in the data
    - For 4-bit mode you will need to clock in each nibble of the command.
3. **`writeChar(char data)`:**
    - Sets the RS pin HIGH.
    - Places the bits corresponding to the character onto the DB pins.
    - Calls your `pulseEnable()` function.
    - The character table of the datasheet shows the encoding. Look at ASCII encoded data to see how characters are encoded as hex/binary values.
4. **`printString(char* text)`:**
    - A loop that accepts an array of characters (a string), breaks it down, and passes each individual character to your `writeChar()` one by one until it hits the null terminator.
    - We may need to develop this together in class since it requires some knowledge of how C handles strings of characters.

And that's it! Yes this is going to be difficult, and yes you can absolutely do it. Take it one step at a time. First, try to just get a blinking cursor to appear which proves your hardware is setup correctly and your commands are working. Then, try to write a single letter 'A' onto the screen. Once you have that, then you can finish up by writing the rest of the string function.

You may also find it useful to create a few other functions that can clear the screen, place the cursor to a specific location, or other functions described in the datasheet.